

Modes of Infinity as Modes of Being — Formalization Blueprint

This canvas compiles the current formal framework into a theorem-ready, Lean-friendly research note.

1. Core idea

We model “modes of being” as **determinacy structures**: an object together with an **idempotent** operator that projects any candidate entity to its determinate (fully instantiated) content.

- **Objects**: (X, α) with $\alpha : X \rightarrow X$ and $\alpha \circ \alpha = \alpha$.
- **Morphisms**: maps commuting with determinacy.
- **Embeddings**: injective/mono morphisms.
- **Size invariant**: cardinality of fixed points (the determinate core).

Philosophical payoff:

- Potential processes can have **trivial global core** even when their “limit” suggests infinity.
 - Ontological composition is often **constraint-coupled**, so naïve $\aleph$ -arithmetic can mislead.
-

2. Set-based core (Lean 4 / mathlib)

2.1 Definitions

```
import Mathlib.SetTheory.Cardinal.Basic

universe u

/-- A mode object is a type with an idempotent determinacy operator. -/
structure ModeObj where
  X      : Type u
  α      : X → X
  idem   : ∀ x, α (α x) = α x

/-- Fixed points (determinate core). -/
def Fix (A : ModeObj.{u}) : Type u :=
  { x : A.X // A.α x = x }

/-- Cardinal-valued size invariant. -/
def F (A : ModeObj.{u}) : Cardinal :=
  #(Fix A)
```

```

/-- Mode morphisms commute with determinacy. -/
structure ModeHom (A B : ModeObj.{u}) where
  f      : A.X → B.X
  comm   : ∀ x, f (A.α x) = B.α (f x)

```

2.2 Induced map on fixed points

```

/-- A mode morphism induces a map on fixed points. -/
def FixMap {A B : ModeObj.{u}} (h : ModeHom A B) : Fix A → Fix B :=
  fun x =>
    ⟨h.f x.1, by
      have := h.comm x.1
      -- use that x is fixed: A.α x = x
      simpa [x.2] using this.symm⟩

```

2.3 Theorem 1: embedding monotonicity (Set case)

If the underlying map is injective, the induced map on fixed points is injective, hence cardinal monotonicity.

```

theorem FixMap_injective
  {A B : ModeObj.{u}} (h : ModeHom A B)
  (hinj : Function.Injective h.f) :
  Function.Injective (FixMap h) := by
  intro x y hxy
  apply Subtype.ext
  apply hinj
  exact congrArg Subtype.val hxy

/-- Monotonicity of F under injective mode morphisms. -/
theorem F_monotone_of_injective
  {A B : ModeObj.{u}} (h : ModeHom A B)
  (hinj : Function.Injective h.f) :
  F A ≤ F B := by
  exact Cardinal.mk_le_of_injective (FixMap h) (FixMap_injective h hinj)

```

3. Ontological composition with constraints

The non- λ -arithmetic phenomenon is modeled by **constraint-coupled composition**:

- Start with product $X \times Y$.
- Impose a law/interaction constraint $K \subseteq X \times Y$.

- Require the constraint be **closed under determinacy**.

3.1 Constrained tensor

```

/-- A constraint K on X×Y, closed under determinacy. -/
structure Constraint (A B : ModeObj.{u}) where
  K      : Set (A.X × B.X)
  closed : ∀ p, p ∈ K → (A.α p.1, B.α p.2) ∈ K

/-- Constrained tensor: subtype of X×Y satisfying K, with induced determinacy. -/
def constrainedTensor (A B : ModeObj.{u}) (C : Constraint A B) : ModeObj.{u} :=
{ X := { p : A.X × B.X // p ∈ C.K }
  α := fun q =>
    ⟨(A.α q.1.1, B.α q.1.2), C.closed q.1 q.2⟩
  idem := by
    intro q
    apply Subtype.ext
    simp [A.idem, B.idem] }

```

3.2 Theorem 2 witness: diagonal constraint on Bool

This is the minimal “teeth” example: each factor has 2 determinate states; the law forces equality, so the composite has only 2 determinate joint states.

```

/-- Bool with α = id (everything determinate). -/
def BoolMode : ModeObj :=
{ X := Bool
  α := id
  idem := by intro b; rfl }

/-- Diagonal (equality) constraint on Bool×Bool. -/
def diagConstraint : Constraint BoolMode BoolMode :=
{ K := fun p => p.1 = p.2
  closed := by
    intro p hp
    -- α=id so closure is immediate
    simp [BoolMode] using hp }

/-- The constrained tensor Bool ⊗_diag Bool. -/
def BoolTensor : ModeObj :=
  constrainedTensor BoolMode BoolMode diagConstraint

-- Goal theorem shape:
-- theorem nonmultiplicative : F BoolTensor < F BoolMode * F BoolMode := ...

```

Note: the final inequality proof can be done by computing both sides as finite cardinals (2 and 4), or by injection/non-injection arguments to avoid arithmetic rewrites.

4. Presheaf upgrade (uniform four-mode semantics)

To model **potential**, **modal**, and **dispositional** structure uniformly, lift the Set-based kernel to presheaves:

$$\mathcal{E} := \mathbf{Set}_\perp^{\mathbf{P}^{\text{op}}}$$

- Objects are functors $\mathbf{P}^{\text{op}} \rightarrow \mathbf{Set}_\perp$.
- Equalizers and monomorphisms are computed **pointwise**.
- Global sections Γ are limits: $\Gamma(X) = \text{Nat}(1, X)$.

4.1 Presheaf-level mode objects

- Objects: $(X, \alpha : X \Rightarrow X)$ with $\alpha \circ \alpha = \alpha$.
 - Fixed points: equalizer presheaf $\text{Fix}(\alpha)$ (pointwise subtype).
 - Size: $F(X, \alpha) = |\Gamma(\text{Fix}(\alpha))|$.
 - **Potential (stages):** $P = (\mathbb{N}, \leq)$.
 - Presheaf $A(n) = [0, n] \sqcup \{\perp\}$ with truncation restrictions.
 - Only coherent global section is constant- $\perp$, so $F = 1$.
 - **Actual (completed):** $P = \mathbf{1}$ or constant presheaf on any P .
 - Global sections recover the completed carrier.
 - **Modal (worlds):** $P = (W, \preceq)$.
 - Global sections enforce world-invariant core (necessities).
 - Total possibility space is pointwise product $(U \sqcup \{\perp\})^W$ (assignments across worlds), typically much larger.
 - **Dispositional (interventions):** $P = (C, \sqsubseteq)$ where $c \sqsubseteq c'$ means “ c' refines c ”.
 - Presheaf encodes outcomes under interventions and restriction forgets detail.
 - Global sections are stable response profiles.
-

5. Proof roadmap

Phase 1 — Set kernel (fastest validation)

1. Implement `ModeObj`, `ModeHom`, `Fix`, `FixMap`.
2. Prove `FixMap_injective` and `F_monotone_of_injective`.
3. Implement `Constraint`, `constrainedTensor`.
4. Prove the diagonal Bool witness of strict non-multiplicativity.

Phase 2 — Presheaves (pointwise lifting)

1. Fix a poset/category `P` and work in `Pop ⇒ Type` (or pointed types).
2. Define presheaf mode objects with idempotent natural transformations.
3. Define fixed-point presheaf pointwise (equalizer).
4. Define `Γ` and `F = #(Γ(Fix α))`.
5. Prove monotonicity by pointwise injectivity + limit preservation.

Phase 3 — Test suite computations

- Compute F for Potential/Actual/Modal/Dispositional examples.
 - Demonstrate “core vs possibility space” separation in the modal case.
 - Demonstrate constraint-driven non-multiplicativity in dispositional composites.
-

6. External references

Lean + mathlib

- Lean 4: <https://lean-lang.org/>
- mathlib documentation: https://leanprover-community.github.io/mathlib4_docs/
- Cardinal basics in mathlib: https://leanprover-community.github.io/mathlib4_docs/Mathlib/SetTheory/Cardinal/Basic.html

Category theory + fixed points / idempotents

- nLab: Idempotent / splitting idempotents: <https://ncatlab.org/nlab/show/idempotent>
- nLab: Presheaf category: <https://ncatlab.org/nlab/show/presheaf>
- nLab: Equalizer: <https://ncatlab.org/nlab/show/equalizer>

Modal / Kripke and presheaf semantics (conceptual background)

- nLab: Kripke semantics: <https://ncatlab.org/nlab/show/Kripke+semantics>
- nLab: Topos-theoretic semantics (entry point): <https://ncatlab.org/nlab/show/topos+theory>

Domain theory / potentiality (optional enrichment)

- Stanford Encyclopedia of Philosophy: Domain Theory (overview): <https://plato.stanford.edu/entries/domain-theory/>

7. Notes on philosophical alignment

- The invariant F measures **determinately instantiated core** (fixed points / coherent global sections), not mere “would-be extension.”
 - Potential infinity becomes **trivial globally** because coherence across all finite stages forces collapse to $\perp$.
 - Non-multiplicativity is driven by **interaction laws** (constraints), not by ad hoc encoding.
-

8. Next concrete deliverable

- A runnable Lean file for Phase 1 (kernel + Bool witness), then
- A second file for Phase 2 (potential presheaf on $\mathbb{N}$ + proof of singleton global sections).

Citizen Gardens © 2025 CC-NC-ND 4.0